Pack 05 · Optimization levers: prefix caching and quantization

Two levers that change the bill: reusing prompt work, and storing or computing in fewer bits. Lessons 03, 04, 06, 11, 14, 22, 40, 46–54, 59. Answer each question out loud in about a minute, then check the card.

Q1 · One dynamic line kills the cache PRIORITY

Lessons: 03, 11, 46, 47

> **INTERVIEWER**
>
> A support assistant sends about 3,300 prompt tokens, 2,900 of them identical across users, but the prefix hit rate is about 1%. The prompt builder puts the request ID and current time on line 2. Why can't the identical tokens after it be reused, what's the fix, and what does it save at about 32 μs per prompt token?

60-second answer

- **Why the identical tokens can't be reused:** in causal attention, each token's K and V depend on **every token before it**. So if one token on line 2 differs, the K and V of every token after it are different too, even where the text is identical.
- **How vLLM enforces this:** it caches in 16-token blocks and uses **chained block hashes**, where each block's hash includes the previous block's hash. That means a miss on the block holding line 2 makes every later block miss as well, which is why the hit rate is about 1%.
- **The fix:** move the request ID and the time **to the end** of the prompt, or take them out of the prompt entirely. The rule is stable content first and variable content last, so the 2,900 shared tokens form an unbroken prefix.
- **What it saves:** before the fix, prefill computes all 3,300 tokens, which takes $3{,}300 \times 32\ \mu s \approx 105$ ms. After it, only the $3{,}300 - 2{,}900 \approx 400$ uncached tokens are computed, which takes $400 \times 32\ \mu s \approx 13$ ms. So each request saves $105 - 13 \approx$ **92 ms**, and since $105 \div 13 \approx 8$, prefill is **about 8× cheaper**.

Key numbers: a cache hit needs an identical prefix from token 1.

Likely follow-up: "What's it worth on an API bill?" Take requests with 3,000 cached system tokens, 200 new input tokens and 300 output tokens, priced per million tokens at $0.55 for input, $0.14 for cached input and $2.19 for output. Without caching, 1,000 requests send 3.2 million input tokens at $0.55 ($1.76) plus 0.3 million output tokens at $2.19 ($0.66), so they cost ≈ **$2.42**. With caching, the 3 million system tokens drop to the cached rate ($0.42), the 0.2 million new tokens stay at $0.55 ($0.11), and output is still $0.66, so the total is ≈ **$1.19**, about half. Output is the same in both cases because output tokens are never discounted.

TRAP

I explained why the ID's own block misses but not why the identical tokens after it miss. The time saved is the difference (105 − 13 ms); the ratio is the speedup.

Q2 · 80% hits, but not 80% faster PRIORITY

Lessons: 47, 04

INTERVIEWER

With an 80% prefix hit rate, the PM expected TTFT to drop 80% and responses to get 5× faster. Instead mean TTFT went 108 → 68 ms, end-to-end time for long answers barely moved, and a batch job with 49% hits got 1.82× input throughput. Explain all three.

60-second answer

- **Why TTFT fell less than the hit rate:** caching removes only the **cached part of prefill**. TTFT also includes fixed costs that caching doesn't touch, like scheduling, prefilling the uncached tail and running the first decode step. That's why TTFT fell from 108 to 68 ms, about 37%, instead of 80%.
- **Why long answers barely moved:** a long answer, say 1,000 output tokens, spends most of its time in **decode**. Caching does nothing for decode, so shaving prefill barely changes the end-to-end time.
- **Why the batch job got 1.82×:** this is Amdahl's law. If a fraction h of prefill is cached, only 1 − h is left to compute, so the best possible prefill speedup is **1 ÷ (1 − hit rate)**. With 49% hits that's 1 ÷ 0.51 ≈ **1.96×**, and the measured 1.82× is close to that ceiling.

Key numbers: max prefill speedup = 1 ÷ (1 − h).

Likely follow-up: "When does prefix caching matter most?" It matters most when prompts share a long prefix and outputs are short, because then prefill is a big share of the work and most of it can be skipped. Typical cases are RAG, agents with big tool schemas, and multi-turn chat.

TRAP

I explained the directions but not why TTFT fell less than the hit rate (fixed costs), and didn't give the 1 ÷ (1 − h) ceiling.

Q3 · Ideal 75%, measured 30% PRIORITY

Lessons: 46, 48

INTERVIEWER

Replaying an hour of real prompts with the server's chained 16-token hashing gives an ideal hit rate of 75%. vLLM's hits ÷ queries over the same hour gives 30%. What causes the gap? And if the replay were low too, how do you find the line to fix?

60-second answer

- **What the replay assumes:** it assumes **unlimited memory, one cache and the same fingerprinting** as the server. So if the ideal rate is high and the measured rate is low, the prompts are fine and the **server** is losing the reuse by breaking one of those assumptions.
- **Memory:** if the cache is too small, blocks get **evicted** before the same prefix comes back, so a repeat that the replay counts as a hit becomes a miss on the server.
- **Fingerprinting:** if the server uses a different **chat template or tokenizer**, or adds a per-tenant cache salt, it hashes different tokens than the replay did, so prefixes that look identical in the replay don't match on the server.
- **One cache:** with **several replicas behind a random load balancer**, each replica sees only a slice of the repeats, so many repeats land on a replica that never saw the first copy.
- **If the replay is low too:** then the prompts themselves break the prefix. Plot a **first-divergence histogram**, which shows the token position where each request first differs from the previous request from the same app. A spike at a low position points at the dynamic line.

Key numbers: none; it's a diagnosis.

Likely follow-up: "Which metric shows eviction?" Look for KV cache usage pinned high while the hit rate falls under load. To confirm it, compare the hit rate at low load and at high load: if it drops only under load, blocks are being evicted before they're reused.

> **TRAP**
>
> I couldn't start. The way in: list what the replay assumes, then ask which assumption the real server breaks.

Q4 · Multi-turn chat, then trimming PRIORITY

Lessons: 14, 40, 48

> **INTERVIEWER**
>
> A chat has a 1,000-token system prompt; each turn adds a 100-token question and a 300-token answer. Over 6 turns, how many prefill tokens without reuse and with a perfect prefix cache? At turn 7 the app starts trimming the oldest message every turn: what happens?

60-second answer

- **Without reuse:** turn n re-sends the system prompt, n questions and $n - 1$ earlier answers, which is $1{,}000 + 100n + 300(n - 1)$ tokens. Over 6 turns that's $1{,}100 + 1{,}500 + 1{,}900 + 2{,}300 + 2{,}700 + 3{,}100 =$ **12,600 tokens**. Each turn re-sends everything before it, so the total grows with the square of the number of turns.
- **With a perfect prefix cache:** the whole previous conversation is cached, **including the model's own answers**, because their K and V were already computed while the answers were being generated. So after turn 1, which computes its 1,100 tokens, each turn computes only the new 100-token question

plus the last partly filled block, which isn't cached. The total is ≈ **1,640 tokens**, about 7× less than 12,600, and it grows linearly.

- **Trimming at turn 7:** dropping the oldest message changes the tokens right after the system prompt. Because the block hashes are chained, **everything after that point misses**, and it happens again on every turn. So each turn recomputes ≈ 2,100 tokens instead of ≈ 100.
- **The better approach:** keep the conversation **append-only**. When you must cut, cut a big chunk rarely, or summarize once, so you pay the full recompute occasionally instead of every turn.

Key numbers: prefix caching turns a conversation's quadratic prefill into linear.

Likely follow-up: "What happens to a finished request's blocks in vLLM?" They go back to the free queue but keep their cached KV, so the next turn can still reuse them until they're overwritten. The deepest blocks are overwritten first, since they're the least shared.

> **TRAP**
>
> "Keep the oldest" was too vague. Say it: any change near the start costs a full recompute, so trim rarely and in bulk.

Q5 · RAG order, replica routing and tenant isolation PRIORITY

Lessons: 47, 48

> **INTERVIEWER**
>
> A RAG prompt is 800 tokens of instructions, 5 passages × 300 tokens, then a 50-token question. Two users get the same passages in a different order. How much is reused, and how do you get near 100%? What happens on 4 replicas behind round-robin, and how do you stop customers A and B from sharing cached K/V?

60-second answer

- **Different passage order:** the prompt is 800 + 5 × 300 + 50 = 2,350 tokens, but the two prompts diverge at the first passage, so only the 800 instruction tokens match. That's 800 ÷ 2,350 ≈ **34%** reused.
- **Getting near 100%:** sort the passages by a **stable key**, such as document ID, so both users see them in the same order. Then they share the instructions and all five passages, 800 + 1,500 = 2,300 tokens, which is 2,300 ÷ 2,350 ≈ **98%**.
- **Four replicas behind round-robin:** a repeat lands on the replica that holds the cached copy only ≈ **1 in 4** times. The fix is to **route by prefix hash**, or by session ID for chats. Routing by request ID doesn't help, because every request has a unique ID.
- **Block granularity:** only **full 16-token blocks** are cached, so for a 1,500-token prompt the reusable part is 16 × ⌊1,500 ÷ 16⌋ = 16 × 93 = **1,488** tokens.
- **Tenant isolation:** mix a per-tenant **cache_salt** into the hash. Then the same text from two different tenants gives different hashes and never matches, and caching stays on.

Key numbers: reusable = 16 × ⌊tokens ÷ 16⌋.

Likely follow-up: "Downside of prefix routing?" A hot prefix sends all its traffic to one replica, which can overload it. So you need load-aware fallbacks that send requests elsewhere when that replica is busy.

> **TRAP**
>
> I said route "by request ID or some hash". A request ID scatters requests just like round-robin; the key must be the shared prefix.

Q6 · RadixAttention, eviction and churn PRIORITY

Lessons: 47, 49

> **INTERVIEWER**
>
> Two prompts share their first 47 tokens: how many does vLLM reuse, and SGLang's RadixAttention? Under memory pressure, what does SGLang evict? And a cache holds 4 prefixes while traffic cycles through 5 (A to E, twice): how many hits under LRU?

60-second answer

- **Reusing a 47-token shared prefix:** vLLM reuses only full 16-token blocks, and 47 tokens hold two full blocks, so it reuses **32** tokens. SGLang's RadixAttention matches token by token, so it reuses all **47**.
- **What SGLang evicts:** it never evicts a node that is **in use**, meaning its reference count is above 0. Among the rest it evicts **LRU leaves** (least recently used) first, which is why shared trunks like the system prompt survive the longest.
- **5 prefixes through a 4-entry LRU:** each prefix is evicted just before it comes back, so **0 of 10 hit**. With one more slot all 5 fit, so the first pass misses and the second pass hits, giving 5 of 10.
- **Cache-aware scheduling:** serving the request with the longest cached prefix first runs a whole group of requests while their prefix is still in the cache. The risk is **starving** requests that share no prefix, because they keep getting pushed back.

Key numbers: a working set just above cache size gives near-zero hits under LRU.

Likely follow-up: "How would you fix churn without more memory?" Group or reorder requests by prefix so each prefix's requests run together, and pin the hottest prefixes so they can't be evicted.

> **TRAP**
>
> I missed the pinning rule (in-use nodes are untouchable) and called starvation "no space for shorter ones".

Q7 · Outliers, AWQ, GPTQ and SmoothQuant PRIORITY

Lessons: 51, 52

> **INTERVIEWER**
>
> Why do a few outlier values make low-bit quantization hard, and how do AWQ, GPTQ and SmoothQuant each work around the problem?

60-second answer

- **Why outliers hurt:** all values in a group share one scale, and **one large value sets the group's scale**. So the grid stretches to fit it, and the small values round to zero. Small groups, like 128 weights or NVFP4's 16, limit the damage because an outlier only spoils its own group.
- **AWQ** (weight-only, 4-bit): about 1% of weight channels matter most, and you find them by their **large activations**. AWQ scales those weights up before rounding so they get a finer grid, and divides the matching activations by the same factor so the output doesn't change.
- **GPTQ** (weight-only): it quantizes the matrix column by column and pushes each column's rounding error onto the **weights not yet quantized**, so later rounding compensates and the errors cancel. It's cheap enough to quantize a 175B model in ≈ 4 GPU-hours.
- **SmoothQuant** (W8A8): here the outliers are in the **activations**. It divides each activation channel by a factor s and multiplies the matching weights by s, so the product is unchanged but both now fit in INT8. That means the **math runs in 8 bits**, which is ≈ 1.5× faster and uses half the memory.

Key numbers: group of 128 with a 16-bit scale costs $4 + 16 \div 128 = 4.125$ bits per weight.

Likely follow-up: "How lossy is each format?" FP8 is essentially lossless. INT8 W8A8 loses about 1–3%. A good 4-bit quantization keeps about 99% of the original score on most benchmarks.

> **TRAP**
>
> I said AWQ quantizes activations too (it's weight-only) and missed what SmoothQuant unlocks (8-bit math).

Q8 · Why 4-bit stops paying off at big batches PRIORITY

Lessons: 50, 51, 53, 59

> **INTERVIEWER**
>
> A 4-bit AWQ model with a Marlin kernel is about 3.5× faster than BF16 at batch 1, about 1.5× at batch 128, and break-even on long prefills. Why? Why would a kernel that expands the whole matrix to 16 bits first be slower than plain BF16? What would you use for a throughput-heavy workload on H100s?

60-second answer

- **Batch 1:** a batch-1 step is limited by **reading weights**. 4-bit weights are 4× fewer bytes than 16-bit ones, so the step gets nearly 4× faster, or ≈ 3.9× once you count the scales ($16 \div 4.125 \approx 3.9$).

- **Batch 128 and long prefills:** a bigger batch reuses each weight for more tokens, so the limit moves from bytes to **math**. The math still runs in 16 bits, and the kernel also has to unpack every tile, so the 4-bit advantage shrinks to about 1.5× and is gone on long prefills.
- **Why expand-first is slower than BF16:** a naive kernel reads the 0.5-byte weight, writes it back as 2 bytes, then reads those 2 bytes again, which is 0.5 + 2 + 2 = **4.5 bytes per weight** versus 2 bytes for plain BF16. Good kernels unpack **in registers** and never write 16-bit weights back to memory.
- **Throughput on H100s:** use **FP8 W8A8**. Both weights and activations go straight into the 8-bit tensor cores, with the scales applied once at the end, so you get ≈ **2× the math** (1,979 versus 989 TFLOPS). So the choice is 4-bit weight-only for a latency-first chatbot and FP8 W8A8 for an overnight batch job.

Key numbers: 4-bit weight-only saves bytes; 8-bit weights + activations saves math. Prefill of 2,000 tokens ≈ 32 ms of math against < 5 ms of weight reading.

Likely follow-up: "A teammate tested 4-bit with 2,000-token prompts and `max_tokens=1`, saw no change, and wants to drop it." That test only measured prefill, which is TTFT, and prefill is compute-bound, so smaller weights barely change it. Benchmark decode instead: use a short fixed prompt and a long answer, and compare TPOT. In general, judge prefill by MFU and decode by MBU, because a decode step running at 1% MFU is normal.

> **TRAP**
> I couldn't start. Ask first: is this workload limited by bytes or by math?

Q9 · "Only 0.8 points, ship it" PRIORITY

Lessons: 06, 22, 54

> **INTERVIEWER**
> The 4-bit model scores 71.2% against BF16's 72.0% on 1,000 questions, and the PM says "only 0.8 points, ship it." What checks would you run first, and why isn't the score enough?

60-second answer

- **Averages hide flips:** some answers can go from right to wrong while others go from wrong to right, and the score stays the same. Microsoft found accuracy moving by 2% or less while up to 13.6% of answers flipped.
- **Distance from the original:** measure how often the top token matches and the KL divergence against the BF16 model. This needs no labels, runs in minutes, and catches broken conversions.
- **Noise floor:** run the baseline against itself in the same engine under load, because batching alone changes some answers. Only flips above that floor count against the 4-bit model.
- **Paired error bars:** each score on its own is ±2.8 points, which is wider than the 0.8 gap, so compare question by question instead. With 34 questions going from wrong to right and 42 going from right to wrong, the difference is −0.8 ± 1.7 points. That's not significant, but you should still read the 42.
- **Your own product:** judge a few hundred real prompts, with your template and lengths, side by side. Decide using a rule written **before** the run.

Key numbers: $\pm 1.96 \times \sqrt{p(1-p) \div n} \approx \pm 2.8$ at $n = 1{,}000$, $p \approx 0.72$.

Likely follow-up: "Temperature 0 makes it deterministic, so why a noise floor?" Because batch size changes the arithmetic, the same prompt can give different outputs depending on how it's batched. In one test, 1,000 runs of one prompt at $T = 0$ gave 80 different completions.

> **TRAP**
> I named only flips and the product eval, and missed the paired error bars.

Q10 · "4-bit loses 10 points" vs 98.9% recovered PRIORITY

Lessons: 51, 54

> **INTERVIEWER**
> A blog post says 4-bit Llama 3.1 loses 10 points on coding; Red Hat's 500,000-evaluation study found 4-bit recovers 98.9% on HumanEval. What explains the gap, and does 98.9% mean users get the same answers?

60-second answer

- **Mismatched setup:** if the blog used a **mismatched chat template** or system prompt, the model was hurt by the setup, not by quantization.
- **Over-sensitive scoring:** exact-match or format checks mark a correct but reworded answer as wrong, so the score drops even when the answer is right. Small evals add noise on top of that.
- **Recovery isn't sameness:** 98.9% recovery compares average scores, so many individual answers can still flip while the average holds. To know whether users get the same answers, check flips against the noise floor and run your own prompts.

Key numbers: 8-bit > 99% recovered; 4-bit 98.9% on HumanEval (Red Hat, Llama 3.1).

Likely follow-up: "Your eval shows a big drop. First thing you check?" I'd check the chat template and the scoring script first, before blaming the weights.

> **TRAP**
> I couldn't start. Before blaming the model, ask whether the test itself was set up correctly.

Build it yourself

- `exercises/02_prefill_tokens.py`: counts fleet prefill tokens for round-robin versus prefix-aware routing across 4 replicas. Run `uv run exercises/02_prefill_tokens.py`.